

Dinamik Programlama Algoritmalarında Zaman ve Enerji Karmaşıklığı Karşılaştırması

Şevval DEMİR
Bilgisayar Mühendisliği

Eda Nur TEKLİK
Bilgisayar Mühendisliği

Ahsen Nur ALDAŞ
Bilgisayar Mühendisliği

Tuğba KORKUT
Yazılım Mühendisliği

Özet — Bu çalışmada, algoritmaların performans analizine enerji tüketimi boyutunu dâhil eden zaman ve enerji karmaşıklığı kavramları ele alınmıştır. Geleneksel zaman ve bellek karmaşıklığı ölçütlerinin modern bilişim sistemleri için yetersiz kalması, özellikle bellek erişimi yoğun algoritmalarda enerji tüketiminin ayrı bir analiz kriteri olarak incelenmesini gerekli kılmaktadır. Bu bağlamda dinamik programlama tabanlı algoritmaların enerji davranışları, teorik bir çerçeve içerisinde değerlendirilmiştir. Çalışmada Bellman–Ford ve Floyd–Warshall algoritmaları, grafik tabanlı dinamik programlama örnekleri olarak incelenirken; 0–1 Knapsack problemi tablo tabanlı dinamik programlama yaklaşımının temsilcisi olarak ele alınmıştır. Her bir algoritma için zaman karmaşıklığı ile bellek erişim yoğunluğunun enerji tüketimine olan etkisi matematiksel modellerle açıklanmış ve enerji karmaşıklığının algoritma tasarımında bağımsız bir performans ölçütü olarak değerlendirilmesi gerektiği ortaya konmuştur.

Anahtar Kelimeler — Energy Complexity, Dynamic Programming, Green Computing, Bellman–Ford Algorithm, Floyd–Warshall Algorithm, Knapsack Problem, Algorithm Analysis

I. GİRİŞ

İklim değişikliği ve sonucunda ortaya çıkan kaygılar, çeşitli sektörleri sürdürülebilirlik uygulamalarını benimsemeye zorlamıştır. Ara hedef olarak Avrupa Birliği 2030 yılına kadar sera gazı emisyonlarının en az %55 oranında azaltılmasını belirlemiş; 2050 yılına kadar da iklim nötrlüğüne ulaşmayı taahhüt etmiştir. Bu küresel eğilimle uyumlu olarak, mühendislik de sürdürülebilirlik gibi acil bir sorundan yalıtılmış kalamaz. [1]

Bilgi ve İletişim Teknolojileri (Information and Communication Technology, ICT) sektörü, sürdürülebilirlik çabalarında önemli bir etkiye sahiptir. Yalnızca ICT sektörünün Avrupa Birliği’ndeki toplam elektrik tüketiminin yaklaşık %8’ini oluşturduğu göz önüne alındığında, 2050 hedeflerine ulaşmak için bu alanın mutlaka ilgi odağı olması gerektiği açıktır. [1]

1.1 Yeşil Bilişim (Green Computing) ve Sürdürülebilirlik

Yazılım, çoğunlukla soyut bir mantıksal yapı olarak ele alınmış; performans değerlendirmesi zaman ve bellek karmaşıklığı ölçütleriyle sınırlandırılmıştır. Ancak iklim

değişikliği, artan enerji maliyetleri ve büyük ölçekli bilişim altyapılarının yaygınlaşması, yazılımın çevresel etkileri de dikkate almasını zorunlu hâle getirmiştir. Bu çerçevede, yazılımın yalnızca algoritmik bir soyutlama değil, doğrudan fiziksel enerji tüketimine etki eden bir sistem bileşeni olduğu gerçeği ön plana çıkmıştır. Özellikle büyük ölçekli veri merkezlerinin yaygınlaşmasıyla birlikte yazılımın enerji üzerindeki etkisi daha görünür hâle gelmiştir. Bulut bilişim altyapıları; milyonlarca sunucu, yüksek kapasiteli soğutma sistemleri ve kesintisiz güç kaynaklarıyla çalışmakta olup küresel elektrik tüketiminin önemli bir bölümünü oluşturmaktadır. Bu durum, bilişim dünyasında çevresel sorumluluğu odak noktasına alan yeni yaklaşımların doğmasına zemin hazırlamıştır.

Yeşil Bilişim (Green Computing), bilgi teknolojilerinin çevresel etkilerini azaltmayı hedefleyen ve sürdürülebilirlik ilkeleriyle doğrudan ilişkili bir araştırma alanıdır. Bu alan, donanım bileşenlerinin yanı sıra yazılım sistemlerinin enerji tüketimi ve kaynak kullanımı üzerindeki rolünü de kapsamaktadır. [1]

Modern yazılım geliştirme süreçlerinde geliştiricilerin donanım mimarisi üzerindeki doğrudan kontrolü sınırlı olsa da, bu donanımın enerji tüketim profilini belirleyen asıl unsur yazılımın kaynak kullanım biçimidir. Bu durum, yazılım seviyesinde alınan algoritmik ve tasarımsal kararları çevresel sürdürülebilirlik açısından en kritik değişken haline getirmektedir. Bu gereklilikten doğan Yeşil Yazılım Mühendisliği (Green Software Engineering); enerji verimliliğini bir tasarım parametresi olarak süreçlere entegre etmeyi ve mevcut sistemleri daha çevre dostu yapılara dönüştürmeyi hedefleyen stratejik bir disiplindir. [1]

Bu bağlamda yazılımın yalnızca işlevsel doğruluk veya çalışma süresi açısından değil, enerji tüketimi açısından da değerlendirilmesi gerekmektedir. Donanım düzeyindeki enerji optimizasyonları bir temel sunsa da, yazılımın bu kaynakları tetikleme sıklığı ve kullanım yoğunluğu toplam enerji tüketiminin asıl belirleyicisidir. Bu durum, algoritma tasarımını yalnızca bir performans kriteri olmaktan çıkarıp, sürdürülebilir bilişim hedefleri doğrultusunda ele alınması gereken temel bir araştırma alanı haline getirmiştir.

1.2 Enerji Karmaşıklığı

Algoritmaların performans analizi geleneksel olarak zaman ve bellek karmaşıklığı ölçütleri üzerinden yapılmıştır. Ancak modern bilişim sistemlerinde enerji tüketimi, en az bu ölçütler kadar önemli bir performans kriteri hâline gelmiştir. Bu doğrultuda enerji, algoritma analizinde bağımsız bir

maliyet ölçütü olarak ele alınmakta ve enerji karmaşıklığı kavramı ile ifade edilmektedir [2]. Enerji tüketimi, bilgisayar sistemlerinde hem maliyet hem de erişilebilirlik açısından da kritik öneme sahiptir. Elektrik maliyetleri, veri ve hesaplama merkezlerinin bütçeleri üzerinde ciddi bir baskı oluşturmaktadır. Binlerce sunucuyu yöneten Google mühendisleri, güç tüketimi artmaya devam ederse enerji maliyetlerinin donanım maliyetlerini büyük bir farkla aşabileceği konusunda uyarıda bulunmuştur.[2] Bu tür ekonomik ve operasyonel baskılar, algoritmaların yalnızca hız ve doğruluk açısından değil; enerji tüketimi açısından da değerlendirilmesini gerekli kılmıştır. Bu bağlamda enerji karmaşıklığı, bir algoritmanın girdi boyutu n 'ye bağlı olarak çalışması sırasında harcadığı toplam enerjinin asimptotik davranışını tanımlar ve $E(n)$ ile gösterilir. Albers [2], enerji tüketimini zamanla değişen güç fonksiyonunun ($P(t)$) integrali olarak modellemekte ve toplam enerji tüketimini (E) aşağıdaki şekilde ifade etmektedir:

$$E = \int_0^T P(t) dt$$

Burada $P(t)$, zamanın bir fonksiyonu olarak anlık güç tüketimini, T ise algoritmanın toplam çalışma süresini göstermektedir. Pratik analizlerde güç tüketiminin ortalama bir değer etrafında değiştiği varsayımı altında bu ifade, aşağıdaki sadeleştirilmiş biçimde kullanılabilir:

$$E(n) = P_{avg} \times T(n)$$

Bu denklem, enerji tüketimi ile zaman karmaşıklığı arasındaki korelasyonu ortaya koymakla birlikte, enerji karmaşıklığının yalnızca zamanın bir yan ürünü olmadığını da kanıtlamaktadır. Albers [2], enerji ve zaman arasında her zaman doğrusal bir ilişki bulunmadığını; nitekim daha yüksek saat frekansında veya paralel işlem yüküyle çalışan 'daha hızlı' algoritmaların, birim zamanda çok daha yüksek enerji maliyetlerine yol açabileceğini vurgulamaktadır. Bu durum, modern algoritma tasarımında zaman-enerji dengesi (trade-off) kavramını merkezi bir konuma taşımaktadır. Enerji tüketimi yalnızca işlemci yükünden ibaret değildir; bellek hiyerarşisindeki veri transferleri ve giriş/çıkış (I/O) operasyonları da toplam maliyetin belirleyici unsurlarıdır. Örneğin; dinamik programlama gibi yoğun bellek erişimi ve saklama (caching) gerektiren yaklaşımlarda, zaman kazancı sağlansa bile enerji tüketiminin zaman karmaşıklığından bağımsız olarak tırmadığı gözlemlenmektedir. Sonuç olarak enerji karmaşıklığı; algoritmaları sadece hız ekseninde değil, kaynak kullanım verimliliği ve maliyet ekseninde de tartmaya olanak sağlayan, modern bilişimin vazgeçilmez bir ölçütüdür.

II. DİNAMİK PROGRAMLAMA ALGORİTMALARI

Dinamik programlama (Dynamic Programming, DP), karmaşık problemlerin daha küçük alt problemlere bölünerek çözüldüğü ve bu çözümlerin saklanıp (memoization/tabulation) tekrar kullanıldığı bir algoritma

tasarım paradigmasıdır. Özellikle çizelgeleme ve optimizasyon problemlerinde sağladığı zaman avantajıyla öne çıkar. Ancak bu yaklaşım, hesaplama süresini kısaltırken karşılığında yoğun bir bellek kullanımı talep eder.

Enerji karmaşıklığı açısından bakıldığında dinamik programlama algoritmalarının maliyeti sadece işlemci döngüleriyle değil, asıl olarak tablolara yapılan sık ve yoğun bellek erişimleriyle belirlenir. Bu durum, zaman karmaşıklığı optimize edilmiş bir DP algoritmasının, bellek trafiği nedeniyle geleneksel analizin öngördüğünden daha yüksek bir enerji profili sergilemesine neden olabilir. Dolayısıyla DP, enerji verimliliği araştırmalarında kritik bir 'denek taşı' niteliğindedir.

Bu çalışmada; dinamik programlama prensiplerini farklı problem türleri ve yapısal yaklaşımlarla ele alan Bellman–Ford, Floyd–Warshall ve 0-1 Knapsack algoritmaları analiz edilmektedir. Bu üçlü seçki; tek kaynaklı en kısa yol, tüm çiftler arası en kısa yol ve kombinatorial optimizasyon problemlerini temsil ederek, zaman ve enerji karmaşıklığı arasındaki gerilimli ilişkiyi farklı veri yapıları üzerinden somutlaştırmayı amaçlamaktadır.

2.1 Bellman–Ford Algoritması

Bellman–Ford algoritması, negatif ağırlıklı kenarların bulunduğu grafiklerde tek kaynaklı en kısa yol problemini çözebilen bir algoritmadır. Algoritma, tüm kenarlar üzerinde ardışık gevşetme (relaxation) işlemleri gerçekleştirerek en kısa yolları hesaplamaktadır. Bu süreçte her düğüm için en kısa mesafe bilgisi tekrar tekrar güncellenmektedir. Bellman–Ford algoritmasının zaman karmaşıklığı, düğüm sayısı V ve kenar sayısı E olmak üzere:

$$T(n) = O(V \cdot E)$$

şeklinde ifade edilmektedir. Algoritma, tüm kenarlar üzerinde $V-1$ kez dolaşarak gevşetme işlemi gerçekleştirdiğinden özellikle yoğun grafiklerde hesaplama maliyeti hızla artmaktadır.

Enerji karmaşıklığı açısından Bellman–Ford algoritmasının temel özelliği, tekrarlanan döngü yapısıdır. Her yinelemede tüm kenarların taranması, işlemci üzerinde sürekli hesaplama yükü oluşturmakta ve bellekten mesafe değerlerinin sıkça okunup yazılmasına neden olmaktadır. Bu durum, enerji tüketiminin yalnızca çalışma süresine değil, aynı zamanda bellek erişim sıklığına da bağlı olarak artmasına yol açmaktadır. Kenar sayısı arttıkça toplam enerji tüketiminin doğrusal olmayan biçimde yükselmesi beklenmektedir.

2.2 Floyd–Warshall Algoritması

Floyd–Warshall algoritması, tüm düğüm çiftleri arasındaki en kısa yolları hesaplayan dinamik programlama tabanlı bir algoritmadır. Algoritma, düğümler arası mesafeleri saklayan bir matris üzerinde çalışmakta ve üçlü iç içe döngü yapısı kullanarak mesafe güncellemelerini

gerçekleştirmektedir. Floyd–Warshall algoritmasının zaman karmaşıklığı:

$$T(n) = O(V^3)$$

şeklinde. Bu kübik karmaşıklık, düğüm sayısının artmasıyla birlikte algoritmanın hesaplama maliyetinin hızla yükselmesine neden olmaktadır.

Enerji karmaşıklığı açısından Floyd–Warshall algoritmasının ayırt edici özelliği, geniş matris yapısı ve yoğun bellek erişimidir. Algoritma boyunca mesafe matrisi sürekli olarak okunmakta ve güncellenmektedir. Bu durum, işlemci hesaplamalarının yanı sıra bellek erişimlerinden kaynaklanan enerji tüketimini de önemli ölçüde artırmaktadır. Özellikle büyük ölçekli grafiklerde matrisin önbellek dışına taşması, enerji tüketiminin daha da yükselmesine yol açabilmektedir. Bu nedenle Floyd–Warshall algoritması, zaman karmaşıklığı yüksek olmasının yanı sıra bellek erişim yoğunluğu nedeniyle enerji karmaşıklığı açısından maliyetli bir algoritma olarak değerlendirilmektedir.

2.3. 0-1 Knapsack Algoritması

0–1 Knapsack problemi, sınırlı kapasiteye sahip bir çanta içerisine yerleştirilecek nesnelerin toplam değerini en büyükecek şekilde seçilmesini amaçlayan klasik bir optimizasyon problemidir. Problemin “0–1” olarak adlandırılmasının nedeni, her bir nesnenin ya tamamen seçilmesi ya da hiç seçilmemesidir; nesnelerin bölünmesine izin verilmez.

Her bir nesne için bir ağırlık ve bir değer tanımlıdır. Amaç, çantanın kapasitesini aşmadan toplam değeri maksimum yapan nesne alt kümesini belirlemektir. Problem matematiksel olarak aşağıdaki şekilde ifade edilir.

$$\begin{aligned} \max \sum_{i=1}^n p_i x_i \\ \text{koşuluyla} \sum_{i=1}^n w_i x_i \leq c \\ x_i \in \{0, 1\}, \quad i = 1, \dots, n \quad [3] \end{aligned}$$

Burada p_i nesne i ’nin değeri, w_i ağırlığı ve c çantanın kapasitesini göstermektedir. Karar değişkeni x_i , ilgili nesnenin çantaya alınıp alınmadığını belirtmektedir.

0–1 Knapsack problemi, dinamik programlama yaklaşımı kullanılarak psödo-polinomik zamanda çözülebilmektedir. Bu yaklaşımda problem daha küçük alt problemlere ayrılmakta ve her alt problem için optimal çözüm değeri bir tablo üzerinde saklanmaktadır. Pferschy tarafından tanımlanan dinamik programlama formülasyonuna göre ilk i nesne kullanılarak kapasitesi d olan bir çanta için elde edilebilecek maksimum değer aşağıdaki şekilde tanımlanır:

$$f(d, i) = \max \left\{ \sum_{j=1}^i p_j x_j \mid \sum_{j=1}^i w_j x_j \leq d \right\}$$

[3]

Bu tanım doğrultusunda, her yeni nesne için tablo güncellenerek optimal çözüm elde edilir. Dinamik programlama tabanlı 0–1 Knapsack algoritmasının zaman ve bellek karmaşıklığı, nesne sayısı n ve kapasite c olmak üzere:

$$T(n, c) = O(nc), \quad S(n, c) = O(nc)$$

şeklinde. Bu yöntem, çözümün optimal olmasını garanti etmekte olup 0–1 Knapsack problemi için en yaygın kullanılan kesin çözüm yaklaşımıdır.

III. METODOLOJİ

Bu çalışmada dinamik programlama tabanlı algoritmaların zaman, bellek ve enerji karmaşıklıkları deneysel olarak incelenmiştir. İncelenen algoritmalar Bellman–Ford, Floyd–Warshall ve 0–1 Knapsack algoritmalarıdır. Tüm deneyler Python programlama dili kullanılarak geliştirilen yazılım üzerinden gerçekleştirilmiştir.

3.1 Deneysel Altyapı

Her algoritma için ayrı deneyler geliştirilmiş ve deneyler aynı ölçüm metodolojisi kullanılarak yürütülmüştür. Bellman–Ford ve Floyd–Warshall algoritmaları için rastgele yönlü ve ağırlıklı grafikler üretilmiş, 0–1 Knapsack algoritması için ise rastgele oluşturulan ağırlık–değer çiftleri kullanılmıştır. Graf tabanlı algoritmalarda, temel girdi parametresi düğüm sayısı n olup kenar yoğunluğu deneylerde sabit tutularak kontrol edilmiştir. 0–1 Knapsack algoritmasında ise problem boyutu öğe sayısı üzerinden tanımlanmıştır.

Her deney birden fazla kez tekrarlanmış ve elde edilen ölçüm değerleri csv formatında saklanmıştır.

3.2 Zaman Ölçümü

Algoritmaların çalışma süresi, yüksek hassasiyetli zaman ölçümü sağlayan **time.perf_counter()** fonksiyonu kullanılarak ölçülmüştür. Ölçüm yalnızca algoritmanın kendisini kapsamakta olup veri üretimi, dosya yazma ve görselleştirme işlemleri ölçüme dahil edilmemiştir. Çalışma süresi, zaman karmaşıklığının deneysel karşılığı olarak değerlendirilmiştir.

3.3 Bellek Ölçümü

Bellek kullanımı iki farklı parametre üzerinden incelenmiştir. İlk olarak, algoritma çalıştırılmadan önceki ve sonraki süreç bazlı bellek kullanımı ölçülmüş ve bellek farkı hesaplanmıştır. Bu ölçüm, işletim sistemi seviyesinde süreç belleğini izleyen **psutil** kütüphanesi kullanılarak gerçekleştirilmiştir. İkinci olarak, algoritma çalışması

sırasında ulaşılan maksimum bellek kullanımı **tracemalloc** kütüphanesi yardımıyla ölçülmüş ve tepe bellek kullanımı (peak memory) ölçümü elde edilmiştir. Bu ölçüm, algoritmanın bellek talebinin en yüksek olduğu noktayı göstermektedir.

3.4 Enerji Ölçümü ve Enerji Karmaşıklığı

Enerji karmaşıklığı iki farklı yaklaşımla ele alınmıştır: Kuramsal enerji karmaşıklığı, verilen tanıma uygun olarak çalışma süresi üzerinden modellenmiştir. Ortalama güç tüketiminin sabit olduğu varsayımı altında enerji karmaşıklığı, aşağıdaki şekilde ifade edilmiştir:

$$E(n) = P_{avg} \times T(n)$$

Bu varsayım altında enerji karmaşıklığı, çalışma süresi ile doğru orantılı kabul edilmiştir.

Deneyisel enerji tüketimi ise **CodeCarbon** kütüphanesi kullanılarak ölçülmüştür. CodeCarbon, algoritmanın çalışması sırasında oluşan CO₂ emisyonunu ölçerek enerji tüketimine deneyisel bir yaklaşım sunmaktadır. Ölçüm yalnızca algoritmanın çalıştığı süreyi kapsamakta olup yardımcı işlemler ölçüm dışında bırakılmıştır.

3.5 Enerji Etki Sonucu (Energy Impact Score)

Zaman ve bellek kullanımını birlikte değerlendirmek maksadıyla ikincil metrik olan Energy Impact Score tanımlanmıştır. Bu metrik aşağıdaki şekilde hesaplanmıştır:

$$EnergyImpact = T(n) \times MemoryDiff$$

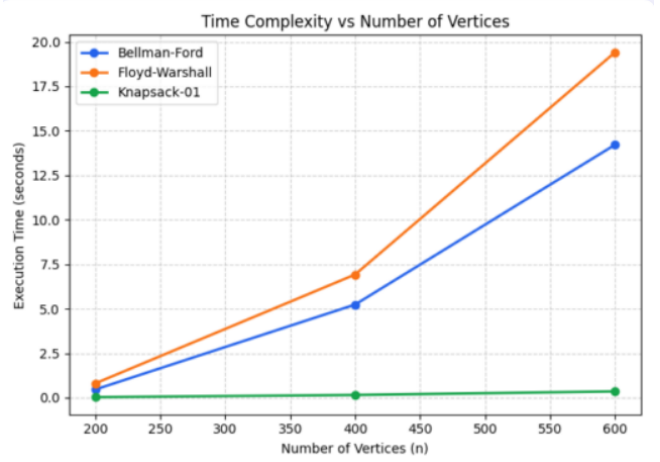
Energy Impact Score doğrudan enerji tüketimini temsil etmemekte olup algoritmaların sistem kaynaklarının üzerindeki etkisini karşılaştırmak amacıyla oluşturulmuştur.

IV. DENEYSEL SONUÇLAR

Bu bölümde Bellman-Ford, Floyd-Warshall ve 0-1 Knapsack algoritmaları için elde edilen deneyisel sonuçlar sunulmaktadır. Deneyler farklı girdi boyutları için birden fazla kez tekrarlanmış ve çalışma süresi, bellek kullanımı ve enerji tüketimi metrikleri ölçülmüştür.

4.1 Zaman Karmaşıklığı

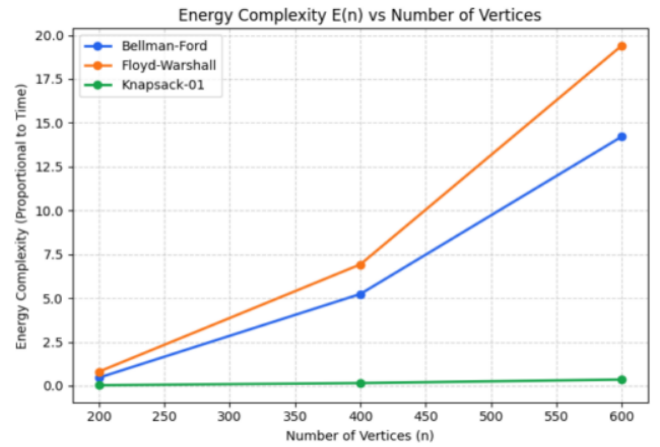
Şekil 1’de algoritmaların düğüm (öge) sayısına bağlı olarak ölçülen çalışma süreleri gösterilmektedir. Girdi boyutu arttıkça Bellman-Ford ve Floyd-Warshall algoritmalarının çalışma sürelerinde artış gözlemlenmiştir. Floyd-Warshall algoritmasının tüm giriş boyutlarında Bellman-Ford algoritmasına kıyasla daha yüksek çalışma süresine sahip olduğu görülmektedir. 0-1 Knapsack algoritması ise ölçülen giriş boyutları için daha düşük çalışma süresi değerleri üretmiştir.



Şekil 1: Farklı girdi boyutlarına göre algoritmaların zaman karmaşıklığı analizi

4.2 Kuramsal Enerji Karmaşıklığı

Şekil 2’de enerji karmaşıklığının $E(n) \propto T(n)$ bağıntısını temel alarak elde edilen sonuçlar yer almaktadır. Bu değerlerin zaman karmaşıklığı analizi ile uyumlu olması, sabit güç varsayımının doğruluğunu teyit etmektedir. Dolayısıyla sabit ortalama güç bağıntısı altında enerji karmaşıklığının zaman karmaşıklığı üzerinden modellenmesinin deneyisel sonuçlarla örtüştüğü görülmektedir.

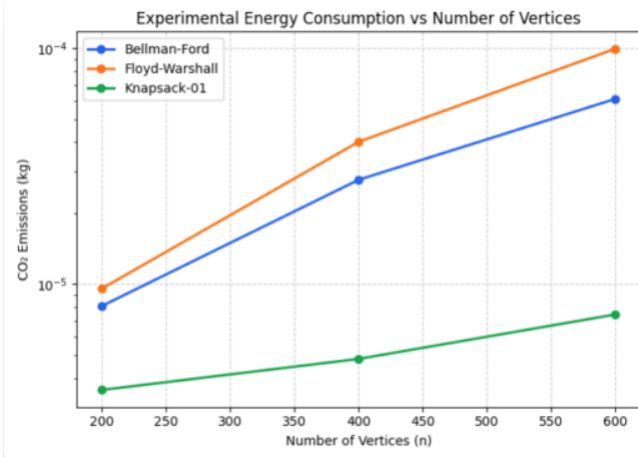


Şekil-2: Farklı girdi boyutlarına göre algoritmaların enerji karmaşıklığı analizi

4.3 Deneyisel Enerji Tüketimi

Şekil 3’te CodeCarbon kullanılarak ölçülen deneyisel enerji tüketimi değerleri verilmiştir. Sonuçlar, giriş boyutu arttıkça algoritmaların CO₂ emisyonlarının da artırdığını göstermektedir.

Floyd-Warshall algoritması en yüksek emisyon değerlerine sahipken Bellman-Ford algoritması orta seviyede bir enerji tüketimine sahiptir. 0-1 Knapsack algoritması ise tüm giriş boyutlarında en düşük deneyisel enerji tüketimini açıkça göstermektedir. Sonuç olarak, algoritmaların hesaplama ve bellek kullanımları deneyisel enerji tüketimini doğrudan etkilemektedir.

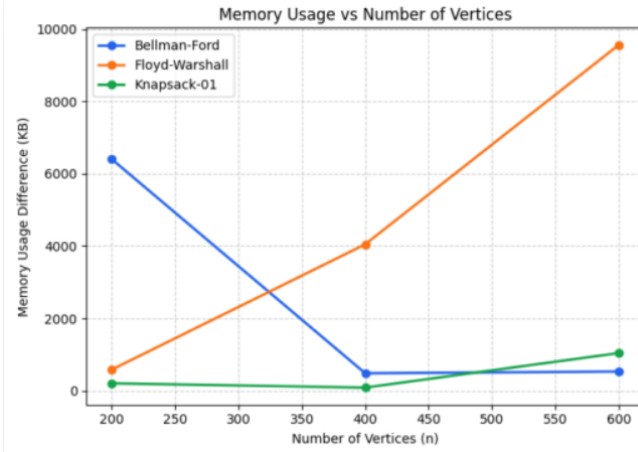


Şekil-3: Algoritmaların ölçülen deneysel enerji tüketimi değerleri

4.4. Bellek Kullanımı

Şekil 4'te algoritmaların ölçülen bellek kullanım farkları sunulmaktadır. Floyd-Warshall algoritmasının bellek kullanımı girdi boyutu arttıkça yükselmektedir. Bellman-Ford algoritması daha sınırlı bellek kullanım değerleri sergilerken, 0-1 Knapsack algoritması daha düşük bellek kullanımıyla öne çıkmaktadır.

Ek olarak, ölçülen tepe bellek kullanımı değerleri, Floyd-Warshall algoritmasının bellek açısından en maliyetli algoritma olduğunu doğrulamaktadır.



Şekil-4: Algoritmaların ölçülen bellek kullanımı değerleri

V. KARŞILAŞTIRMALI DEĞERLENDİRME

Tablo-1: Algoritmaların karşılaştırılması

Algoritma	Zaman Karmaşıklığı/ Maliyeti	Bellek Kullanım Yoğunluğu	Enerji Tüketim
Floyd-Warshall	Çok Yüksek $O(V^3)$	En Yüksek (Matris Tabanlı)	En Yüksek
Bellman-Ford	Orta $O(V * E)$	Düşük/Orta (Kenar Listesi Tabanlı)	Dengeli
0-1 Knapsack	Düşük $O(n * W)$	Düşük	En Düşük

Tablo-1, incelenen üç temel algoritmanın zaman, bellek ve toplam enerji yükü parametreleri arasındaki hiyerarşiyi net bir şekilde ortaya koymaktadır. Elde edilen bulgular; Floyd-Warshall algoritmasının karmaşık veri yapısı ve yoğun döngü gereksinimi nedeniyle her üç kategoride de en yüksek maliyete sahip olduğunu göstermektedir. Buna karşın, 0-1 Knapsack algoritmasının daha dar bir çözüm uzayında, kontrollü bellek erişimleriyle çalışması; onu enerji verimliliği açısından en başarılı seçenek olarak konumlandırmaktadır. Bellman-Ford algoritması ise hesaplama yoğunluğuna rağmen bellek yönetimindeki başarısıyla 'orta düzey' bir enerji profili sergilemiştir. Sonuç olarak, zaman ve bellek metriklerinin kümülatif etkisiyle oluşan Toplam Enerji Yükü, algoritmaların sürdürülebilirlik performansını değerlendirmek için geleneksel karmaşıklık analizlerine göre çok daha kapsamlı bir perspektif sunmaktadır.

VI. SONUÇ

Elde edilen bulgular, yazılımın performans analizinde geleneksel zaman ve bellek ölçütlerinin ötesine geçilerek enerji verimliliğinin de aslı bir parametre olarak kabul edilmesi gerektiğini doğrulamaktadır. Floyd-Warshall algoritmasının sergilediği yüksek maliyet, yoğun hesaplama ve bellek trafiğinin birleştiği durumlarda enerji tüketiminin nasıl bir darboğaza dönüşebileceğini kanıtlar niteliktedir. Öte yandan, Bellman-Ford ve 0-1 Knapsack algoritmalarının daha dengeli ve kontrollü kaynak kullanımı, doğru algoritma seçiminin çevresel sürdürülebilirlik üzerindeki doğrudan etkisini ortaya koymaktadır. Sonuç olarak; algoritmik kararların sadece operasyonel hız değil, fiziksel enerji maliyeti üzerinden de optimize edilmesi, 'Yeşil Yazılım Mühendisliği' prensiplerinin hayata geçirilmesinde temel bir gerekliliktir. Bu çalışma, karmaşıklık analizlerine enerji boyutunun dahil edilmesinin, geleceğin bilişim altyapıları için daha verimli ve çevre dostu sistemlerin inşasına stratejik bir katkı sunacağını göstermektedir.

KAYNAK

- [1] <https://helda.helsinki.fi/server/api/core/bitstreams/e13ea726-c362-4e10-8ef3-2942ee76f333/content>
- [2] <https://dl.acm.org/doi/epdf/10.1145/1735223.1735245>
- [3] <https://link.springer.com/article/10.1007/s006070050042#preview>
- [4] https://www.researchgate.net/profile/Magzhan-Kairanbay/publication/310594546_A_Review_and_Evaluations_of_Shortest_Path_Algorithms/links/5ac49631a6fdec1a5bd06106/A-Review-and-Evaluations-of-Shortest-Path-Algorithms.pdf
- [5] <https://link.springer.com/article/10.1007/s11047-015-9483-8>